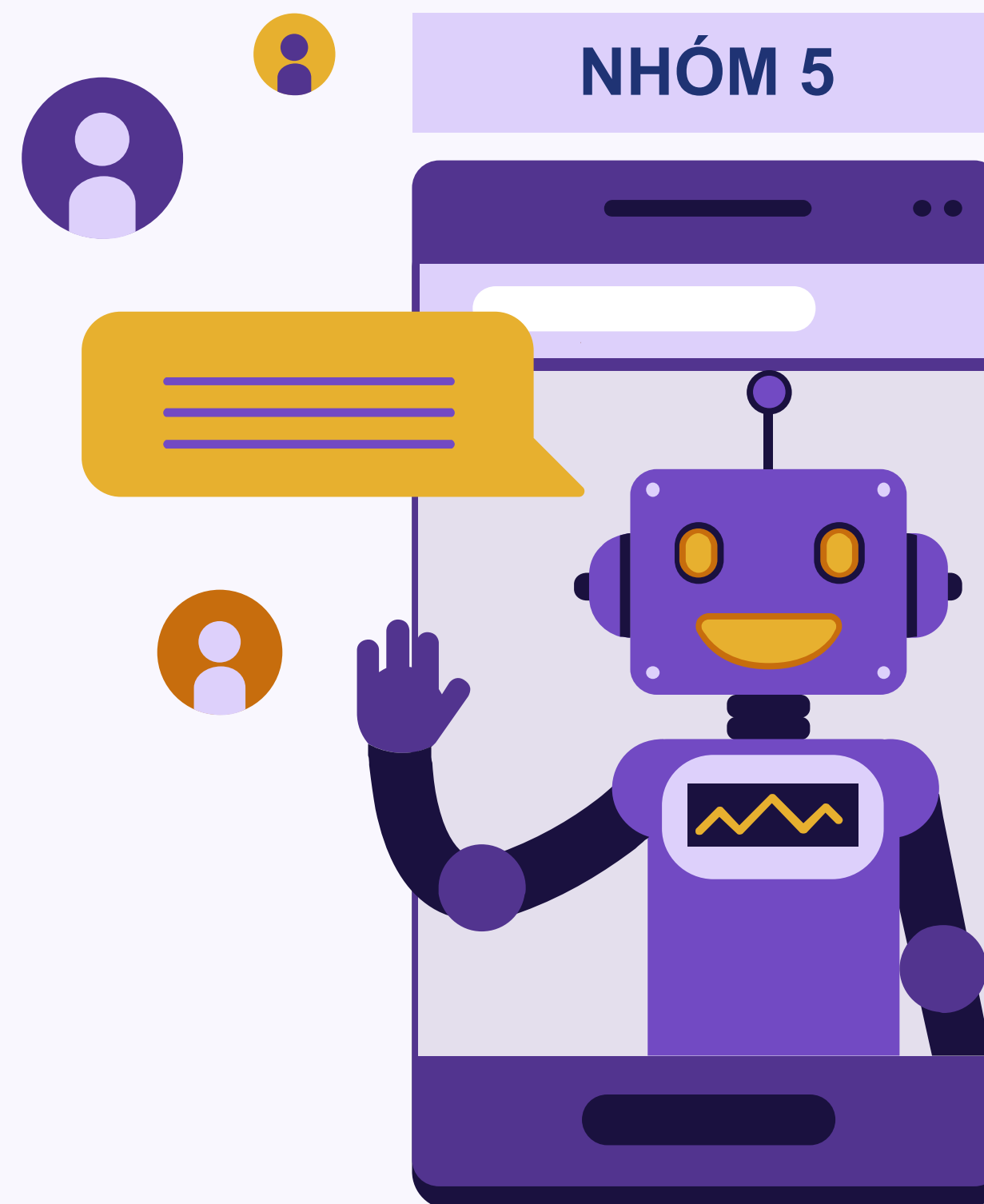




# SPIKING NEURAL NETWORKS HỆ THỐNG CHATBOT AGENTIC TRONG QUẢN LÝ THƯƠNG MẠI

**CS338.Q22 - Nhận dạng**  
*GV: Dương Việt Hằng*  
*Trần Doãn Thuyền*

**23520984 - Phạm Quốc Nam**  
**23520662 - Nguyễn Khang Hy**



# NỘI DUNG TRÌNH BÀY



## Tổng quan về SNN/SpikeGPT:

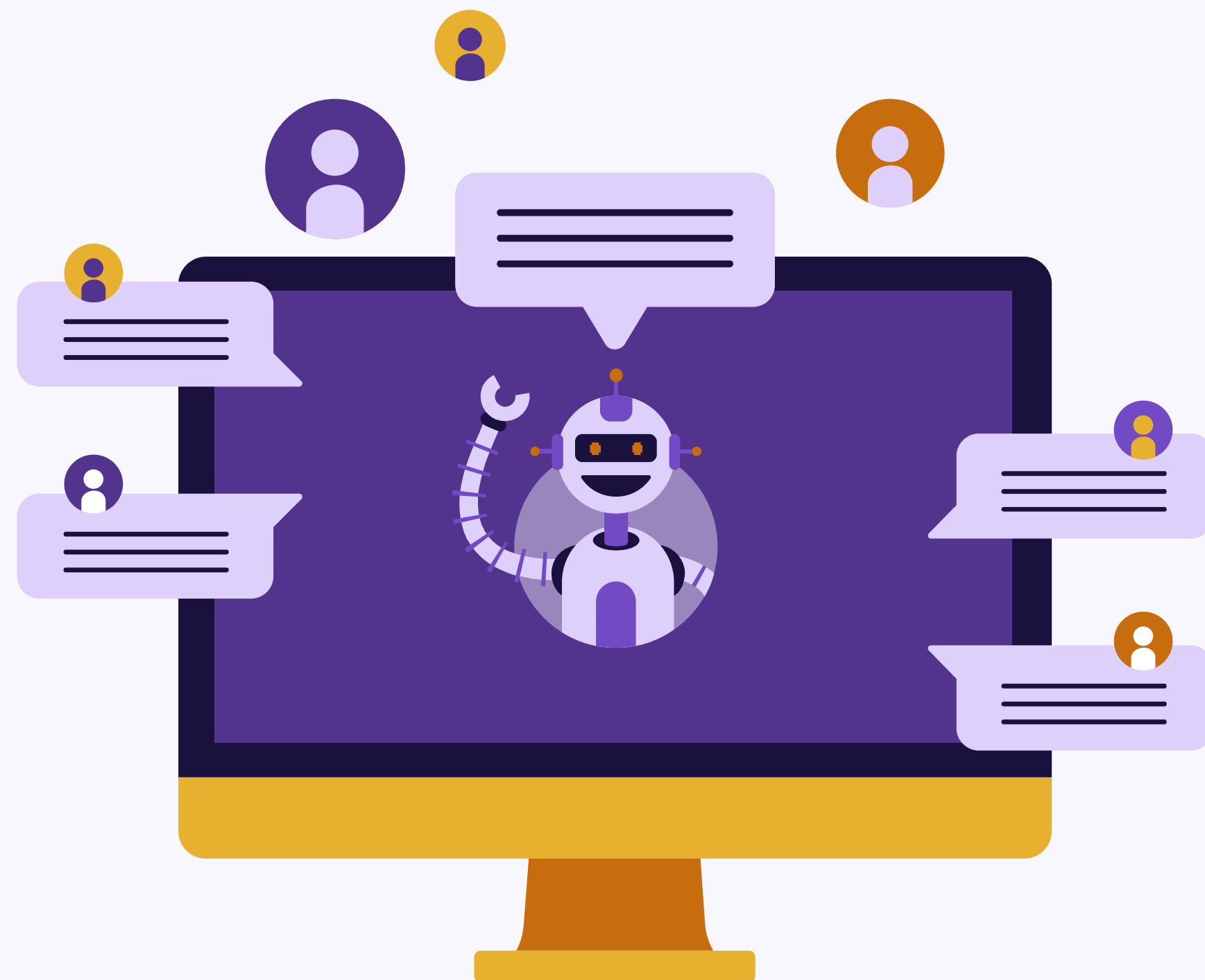
- Rào cản của Transformer và Sự ra đời của SpikeGPT
- Động học Nơ-ron LIF & Surrogate Gradient
- Kiến trúc SpikeGPT: Binary Embedding
- Đột phá Toán học: Spiking RWKV (Time-mixing)
- Spiking Receptance Feed-Forward Network (Channel-mixing)



## Đề án môn học:

- Giới thiệu bài toán: Agentic Chatbot trong Thương mại
- Phương pháp luận & Xây dựng Dữ liệu
- Kết quả Thực nghiệm & Đánh giá
- Kết luận & Hướng phát triển





*SNN for Chatbot*  
**TỔNG QUAN  
VỀ SPIKEGPT**

# 1. RÀO CẢN CỦA TRANSFORMER ĐỐI VỚI SNN



## Sự bất đồng bộ về thời gian

- SNN hoạt động dựa trên cơ chế hướng sự kiện theo từng bước thời gian  $t$ . Trong khi đó, cơ chế Self-Attention yêu cầu quan sát toàn cục toàn bộ chuỗi đầu vào cùng lúc.

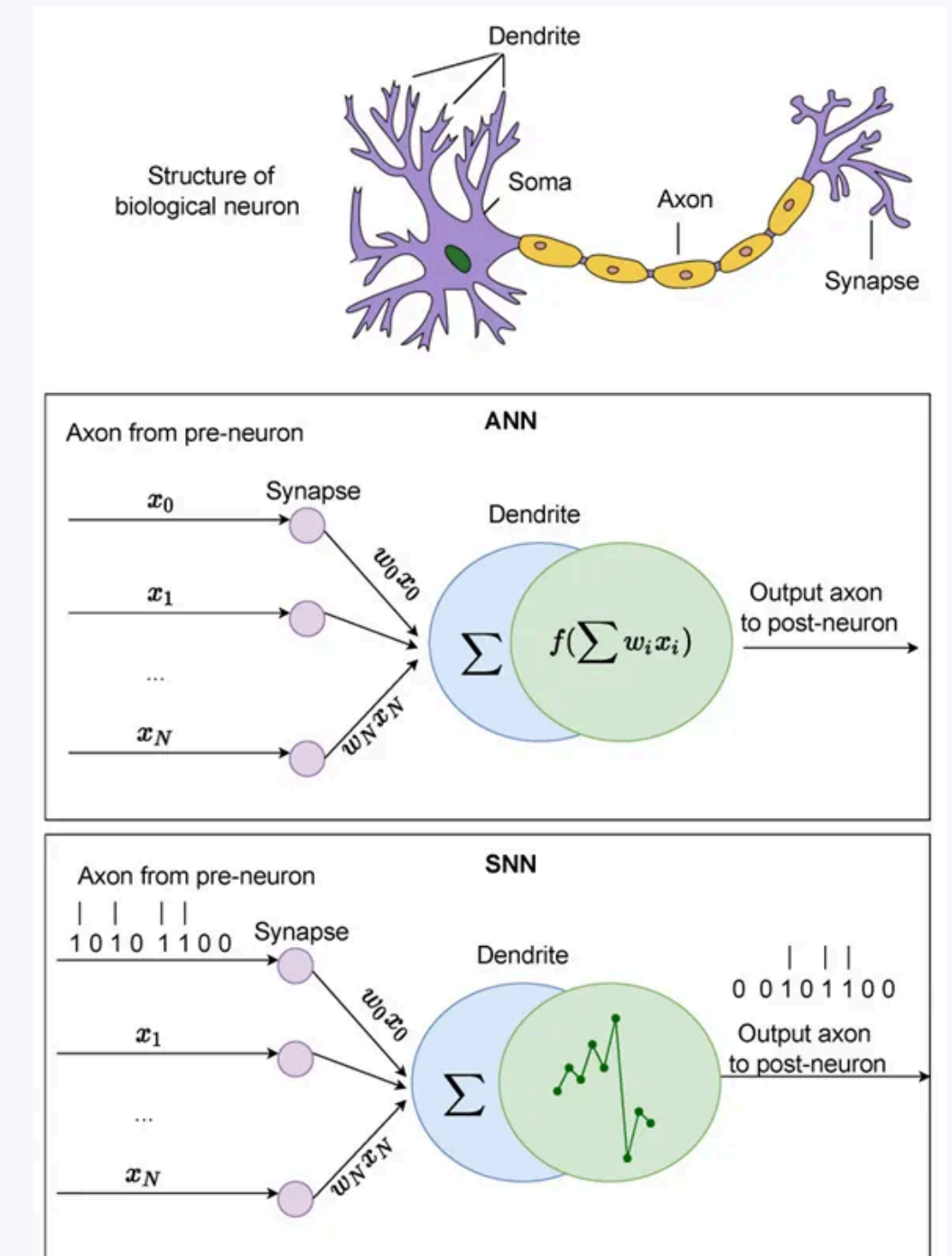
## Hàm Softmax & Ma trận Dấu phẩy động

$$\text{Attention}(Q, K, V) = \text{softmax} \left( \frac{QK^T}{\sqrt{d_k}} \right) V$$

- Softmax đòi hỏi phép tính lũy thừa và chia số thực. Điều này phá vỡ tính chất "chỉ tính toán trên xung nhị phân 0/1" của SNN, làm mất đi lợi thế tiết kiệm năng lượng.

## Độ phức tạp $O(N^2)$

- Phép nhân ma trận  $QK^T$  tạo ra gánh nặng tính toán khổng lồ không thể mở rộng trên phần cứng Neuromorphic.





## 2. LEAKY INTEGRATE-AND-FIRE (LIF) NEURON



- Hoạt động qua 3 giai đoạn (hệ phương trình vi phân rời rạc):

### 1. Cập nhật điện thế màng (Membrane Potential Update):

$$U[t] = H[t] + \beta \left( Y[t] - (H[t - 1] - U_{reset}) \right)$$

### 2. Cơ chế kích hoạt xung (Spike Firing):

$$S[t] = \Theta(U[t] - U_{threshold})$$

### 3. Thiết lập lại sau xung (Reset Mechanism):

$$H[t] = U[t] \cdot (1 - S[t])$$

## 2. LEAKY INTEGRATE-AND-FIRE (LIF) NEURON



$$\begin{cases} U[t] = H[t] + \beta(Y[t] - (H[t - 1] - U_{\text{reset}})) \\ S[t] = \Theta(U[t] - U_{\text{threshold}}) \\ H[t] = U[t] \cdot (1 - S[t]) \end{cases}$$

### Phân tích toán học:

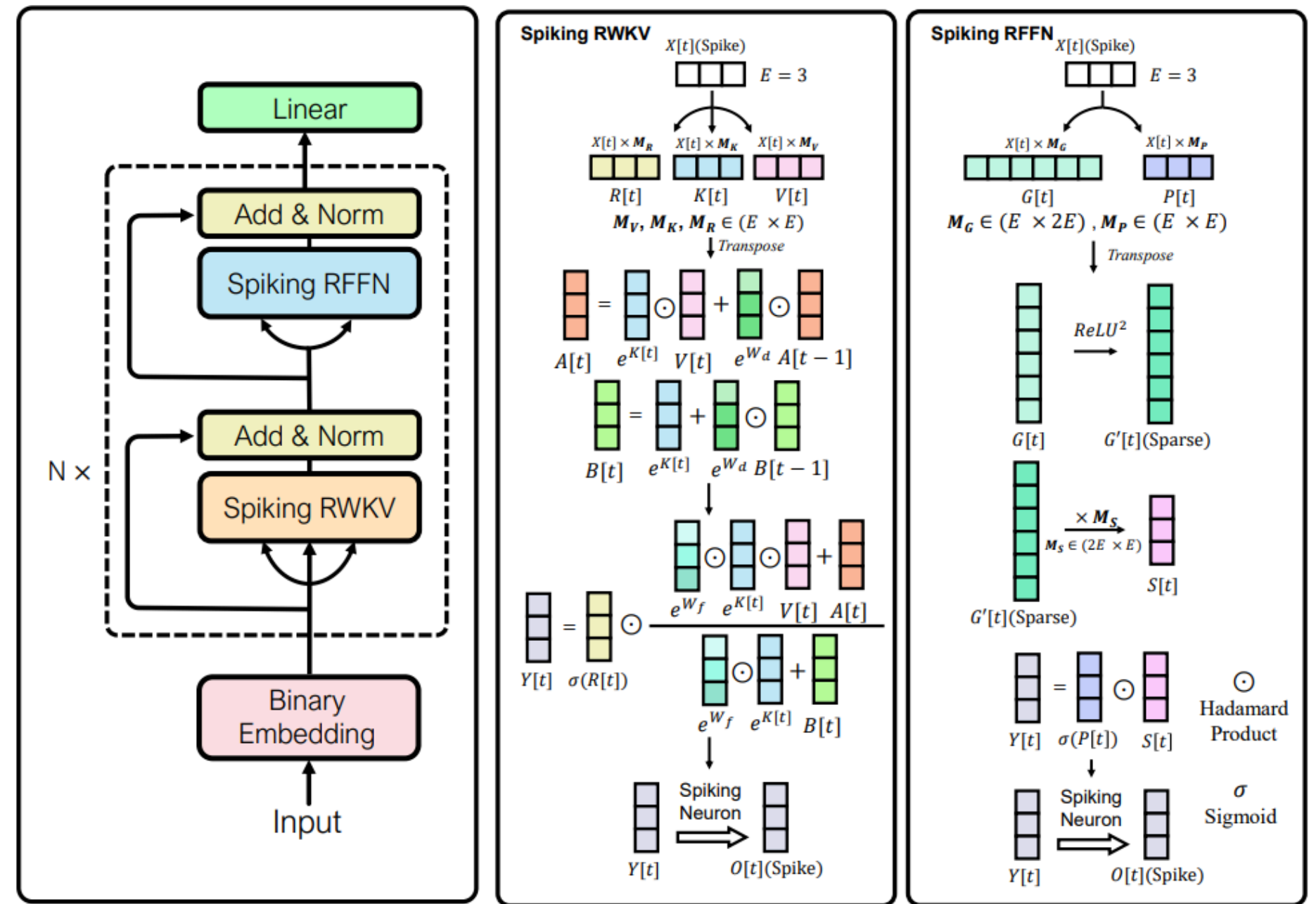
- $U[t]$  là điện thế màng tại thời điểm  $t$ ,  $\beta$  là hệ số suy giảm (decay factor, thiết lập mặc định  $\beta = 0.5$ ), và  $Y[t]$  là tín hiệu đầu vào từ tầng trước (Spiking RWKV).
- $\Theta(\cdot)$  là hàm bước Heaviside (Heaviside step function). Nếu  $U[t]$  vượt ngưỡng  $U_{\text{threshold}} = 1$ , nơ-ron sẽ phát ra xung nhị phân  $S[t] = 1$ .
- Sau khi phát xung ( $S[t] = 1$ ) thì giá trị  $H[t]$  (trạng thái ẩn) sẽ bị triệt tiêu về 0 (do  $U_{\text{reset}} = 0$ ), đây gọi là cơ chế *hard reset* để tránh hiện tượng phát xung liên tục.

### 3. KIẾN TRÚC SPIKEGPT



$$\begin{aligned}
 X_0 &= \text{BE}(I), & I &\in \mathbb{R}^{T \times d}, X_0 \in \mathbb{R}^{T \times d} \\
 X'_l &= \text{SRWKV}(X_{l-1}) + X_{l-1}, & X'_l &\in \mathbb{R}^{T \times d}, l = 1 \dots L \\
 X_l &= \text{SRFFN}(X'_l) + X'_l, & X_l &\in \mathbb{R}^{T \times d}, l = 1 \dots L \\
 Y &= \text{GH}(X_L).
 \end{aligned}$$

- Residual Connections:** Cấu hình **SEW-ResNet** cộng trực tiếp đầu vào  $X_{l-1}$  (vốn là các xung nguyên) với đầu ra của khối. Điều này đảm bảo thông tin cũ không bị mất khi huấn luyện



Mô hình SpikeGPT 216M Tham số

### 3. KIẾN TRÚC SPIKEGPT



- **BINARY EMBEDDING:**

**Mục đích:** Để nhất quán với SNN, văn bản đầu vào không thể là vector số thực. Tầng Nhúng nhị phân chuyển đổi ngay từ ngõ vào thành chuỗi xung nhị phân (0, 1).

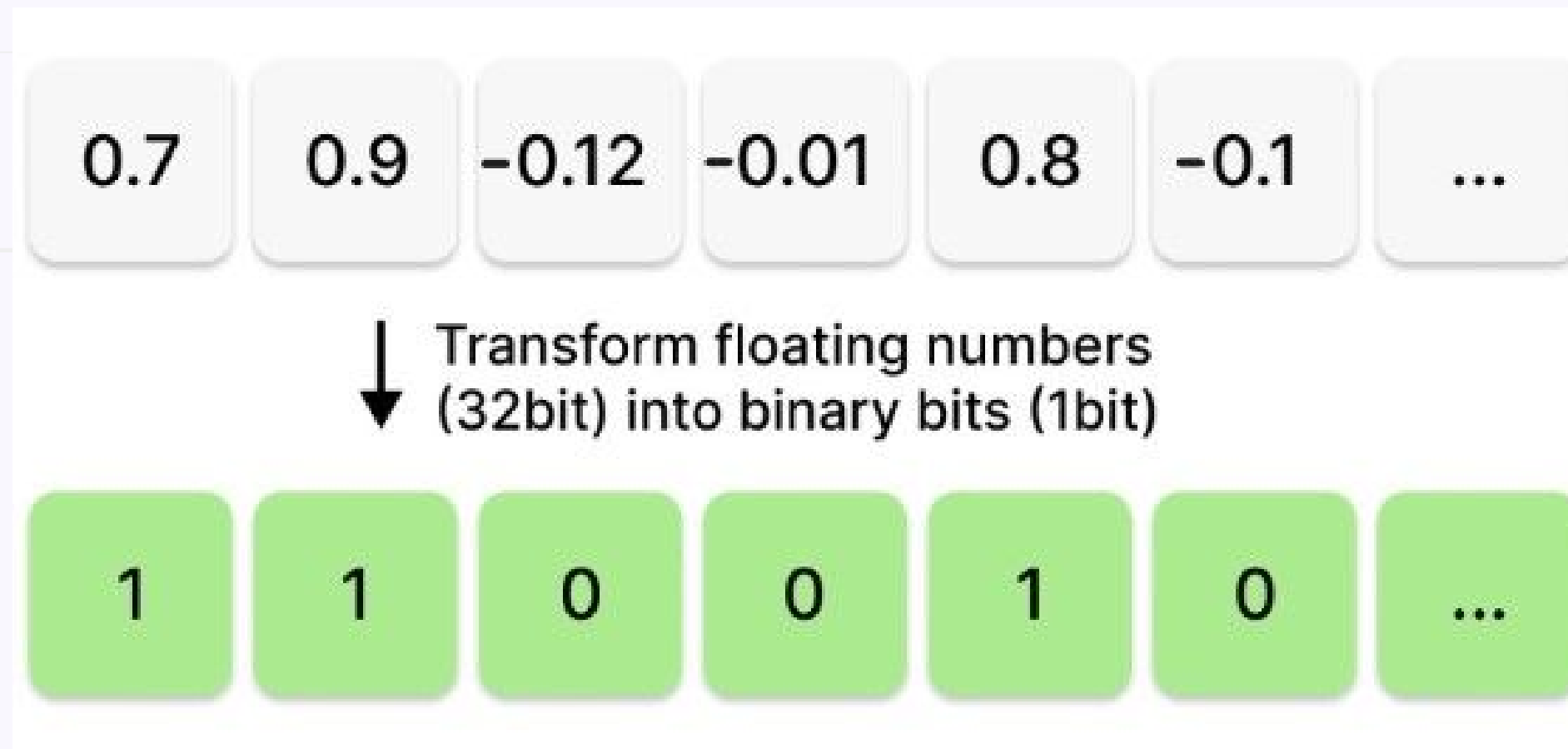
**Pha Lan truyền xuôi (Forward Pass):**

- Áp dụng hàm Heaviside  $\Theta(\cdot)$  ép kiểu cứng:

$$X_0 = \text{BE}(I) = \Theta(I - \text{Threshold})$$

$$I \in \mathbb{R}^{T \times d}, X_0 \in \mathbb{R}^{T \times d}$$

→ Giúp loại bỏ hoàn toàn phép nhân ma trận trong quá trình suy luận ở các lớp sau.



### 3. KIẾN TRÚC SPIKEGPT



#### Pha Lan truyền ngược (Backward Pass):

- Sử dụng **Surrogate Gradient** (arctangent) để có thể cập nhật không gian vector từ vựng (continuous weights) khi mạng học.

Hàm  $\Theta(\cdot)$  có đạo hàm bằng 0 tại mọi nơi, làm cho thuật toán **lan truyền ngược** thất bại.

SpikeGPT giải quyết bằng cách dùng **đạo hàm thay thế (Surrogate Gradient)** trong quá trình lan truyền ngược, cụ thể là đạo hàm của hàm arctangent:

$$\sigma'(x) = \frac{\alpha}{2(1 + (\frac{\pi}{2}\alpha x)^2)}$$

$$\sigma'(x) = \frac{1}{\pi \arctan(\pi x) + \frac{1}{2}}$$

Hoặc dạng tương đương  $\sigma'(x) = \frac{1}{\pi \arctan(\pi x) + \frac{1}{2}}$ . Việc này cung cấp một công cụ ước lượng gradient chệch (biased gradient estimator) nhưng mượt mà, giúp mạng cập nhật trọng số hiệu quả mà không phá vỡ tính chất xung ở chiều truyền xuôi.



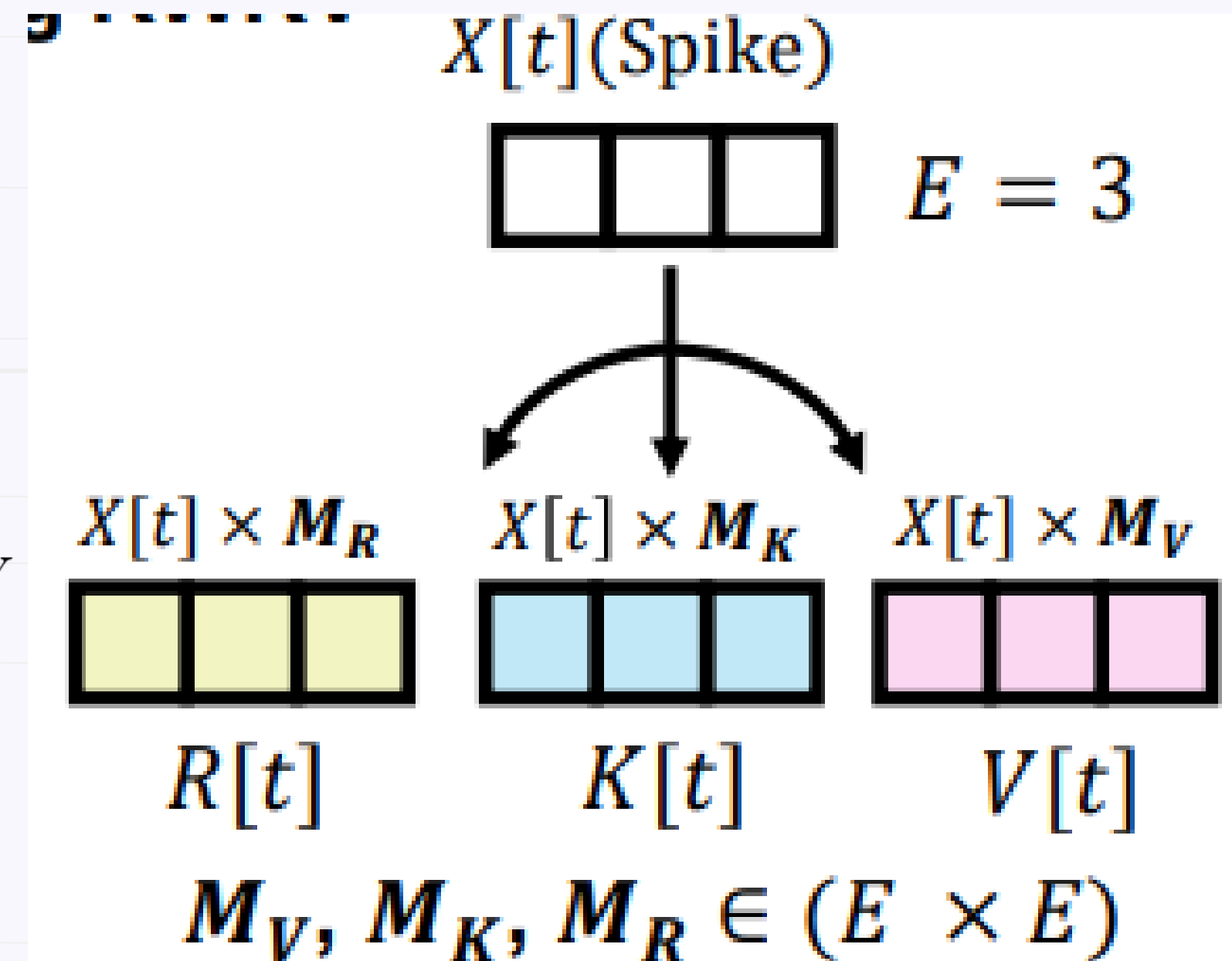
## 4. SPIKING RWKV



### Mô đun Spiking RWKV (SRWKV):

Thay vì phải nhìn toàn bộ chuỗi đầu vào  $\mathbf{X}$  cùng một lúc như **Transformer**, **Spiking RWKV** xử lý dữ liệu theo từng bước thời gian  $t$ , ký hiệu là  $\mathbf{X}[t]$ .

- Tại mỗi bước, nó tính toán 3 thành phần: Receptance ( $\mathbf{R}$ ), Key ( $\mathbf{K}$ ), và Value ( $\mathbf{V}$ ) thông qua các phép biến đổi tuyến tính  $M_R, M_K, M_V$
- Sử dụng  $\mathbf{R}$  (Receptance - Khả năng tiếp nhận) thay thế cho  $\mathbf{Q}$  (Query - Truy vấn) của Attention truyền thống.



## 4. SPIKING RWKV



### Mô đun Spiking RWKV (SRWKV):

Để duy trì được ngữ cảnh mà không cần nhìn lại toàn bộ chuỗi, Spiking RWKV sử dụng các Trạng thái ẩn (Hidden states) **A** và **B**. Chúng đóng vai trò như bộ nhớ dài hạn của mạng:

$$A[t] = \exp(K[t]) \odot (V[t] + \exp(W_d) \odot A[t - 1])$$

$$B[t] = \exp(K[t - 1]) + \exp(W_d) \odot B[t - 1]$$

Hệ số phân rã  $W_d$  (Decay vector): Đây là bộ lọc quên. Khối  $\exp(W_d)$  quyết định xem hệ thống sẽ giữ lại bao nhiêu thông tin từ quá khứ  $A[t - 1]$  và  $B[t - 1]$  khi bước sang thời điểm mới.



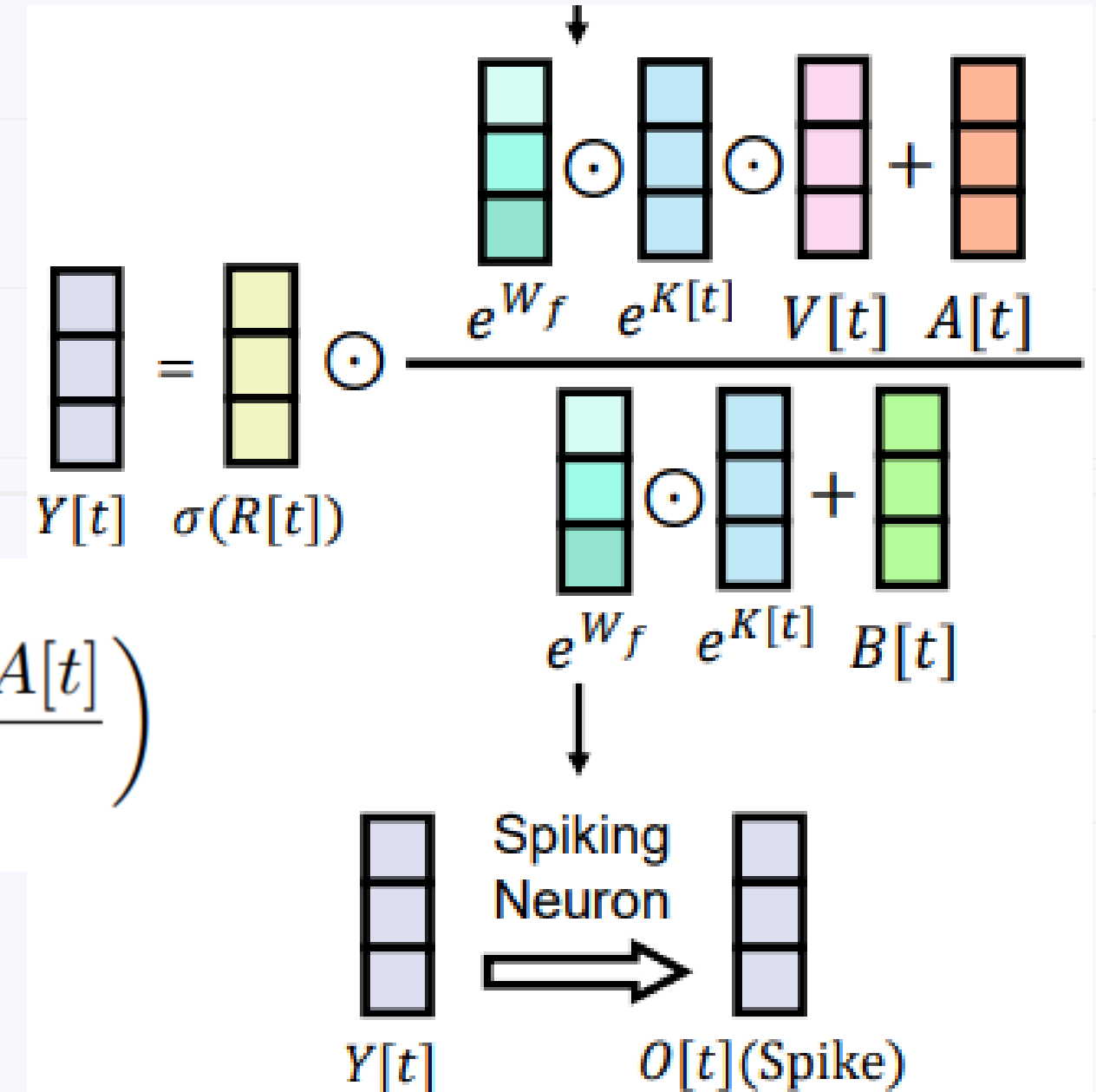
## 4. SPIKING RWKV



### Mô đun Spiking RWKV (SRWKV):

Đầu ra tại bước thời gian tiếp theo  $Y[t + 1]$  được tính toán bằng sự kết hợp giữa quá khứ và hiện tại:

$$Y[t + 1] = \mathcal{SN} \left( \sigma(R[t]) \odot \frac{\exp(W_f) \odot \exp(K[t]) \odot (V[t]) + A[t]}{\exp(W_f) \odot \exp(K[t]) + B[t]} \right)$$



## 4. SPIKING RWKV



Mô đun Spiking RWKV (SRWKV):

$$Y[t + 1] = \mathcal{SN} \left( \sigma(R[t]) \odot \frac{\exp(W_f) \odot \exp(K[t]) \odot (V[t]) + A[t]}{\exp(W_f) \odot \exp(K[t]) + B[t]} \right)$$

- $W_f$ : Trọng số đặc biệt dùng để nhấn mạnh (weighting) thông tin của chính bước hiện tại. Nó giúp mô hình chú ý nhiều hơn vào từ vừa xuất hiện.
- $A[t]$ : Đây là trạng thái ẩn mang theo toàn bộ giá trị tích lũy từ quá khứ (từ bước  $t - 1$  trở về trước).
- Ý nghĩa: Tổng hợp thông tin quan trọng của hiện tại và toàn bộ thông tin quan trọng của quá khứ.

## 4. SPIKING RWKV



Mô đun Spiking RWKV (SRWKV):

$$Y[t + 1] = \mathcal{SN} \left( \sigma(R[t]) \odot \frac{\exp(W_f) \odot \exp(K[t]) \odot (V[t] + A[t])}{\exp(W_f) \odot \exp(K[t]) + B[t]} \right)$$

- Cộng dồn các trọng số của hiện tại và quá khứ  $B[t]$
- Ý nghĩa: chuẩn hóa hệ như cách hàm softmax đảm bảo tổng các xác suất luôn bằng 1

## 4. SPIKING RWKV



Mô đun Spiking RWKV (SRWKV):

$$Y[t+1] = \mathcal{SN} \left( \sigma(R[t]) \odot \frac{\exp(W_f) \odot \exp(K[t]) \odot (V[t]) + A[t]}{\exp(W_f) \odot \exp(K[t]) + B[t]} \right)$$

$R[t]$  (**Receptance**): Đóng vai trò tương tự như **Query** ( $Q$ ) trong Attention, nhưng thay vì dùng để tìm kiếm, nó dùng để quyết định sự tiếp nhận. Hàm **sigmoid**  $\sigma(\dots)$  sẽ ép giá trị của  $R[t]$  về khoảng từ 0 đến 1. **Phép nhân từng phần tử** ( $\odot$ ) sẽ lấy kết quả từ **khối trộn thông tin** nhân với chiếc cổng này. Nếu  $\sigma(R[t])$  **gần bằng 0**, thông tin sẽ bị chặn lại. Nếu **gần bằng 1**, thông tin được phép đi qua toàn bộ.

## 4. SPIKING RWKV



Mô đun Spiking RWKV (SRWKV):

$$Y[t + 1] = \mathcal{SN} \left( \sigma(R[t]) \odot \frac{\exp(W_f) \odot \exp(K[t]) \odot (V[t]) + A[t]}{\exp(W_f) \odot \exp(K[t]) + B[t]} \right)$$

Toàn bộ **cụm tính toán bên trong ngoặc đơn** trả về các **số thực liên tục (floating-point numbers)**

Lớp  $\mathcal{SN}$  sẽ nhận các số thực này và ép thành các **xung nhị phân (0 hoặc 1)**

Đầu ra cuối cùng  $Y[t + 1]$  sẽ là một **vector chỉ toàn các giá trị 0 và 1**, sẵn sàng để được truyền tới lớp mạng tiếp theo một cách **cực kỳ tiết kiệm năng lượng**



## 4. SPIKING RWKV



- **Xử lý Dữ liệu Tuần tự (Bước thời gian):**
  - SRWKV chỉ nhận đầu vào  $X[t]$  ứng với từng bước thời gian  $t$ , thay vì toàn bộ chuỗi  $X[t]$
  - Thiết kế này hoàn toàn tương thích với cấu trúc của mạng SNN.
- **Sử dụng Phép nhân Từng phần tử (Element-wise Products):**
  - SRWKV đã loại bỏ hoàn toàn các phép nhân ma trận tốn kém.
  - Thay vào đó, SRWKV sử dụng các phép toán nhân từng phần tử để trao đổi thông tin giữa các token.



## 4. SPIKING RWKV



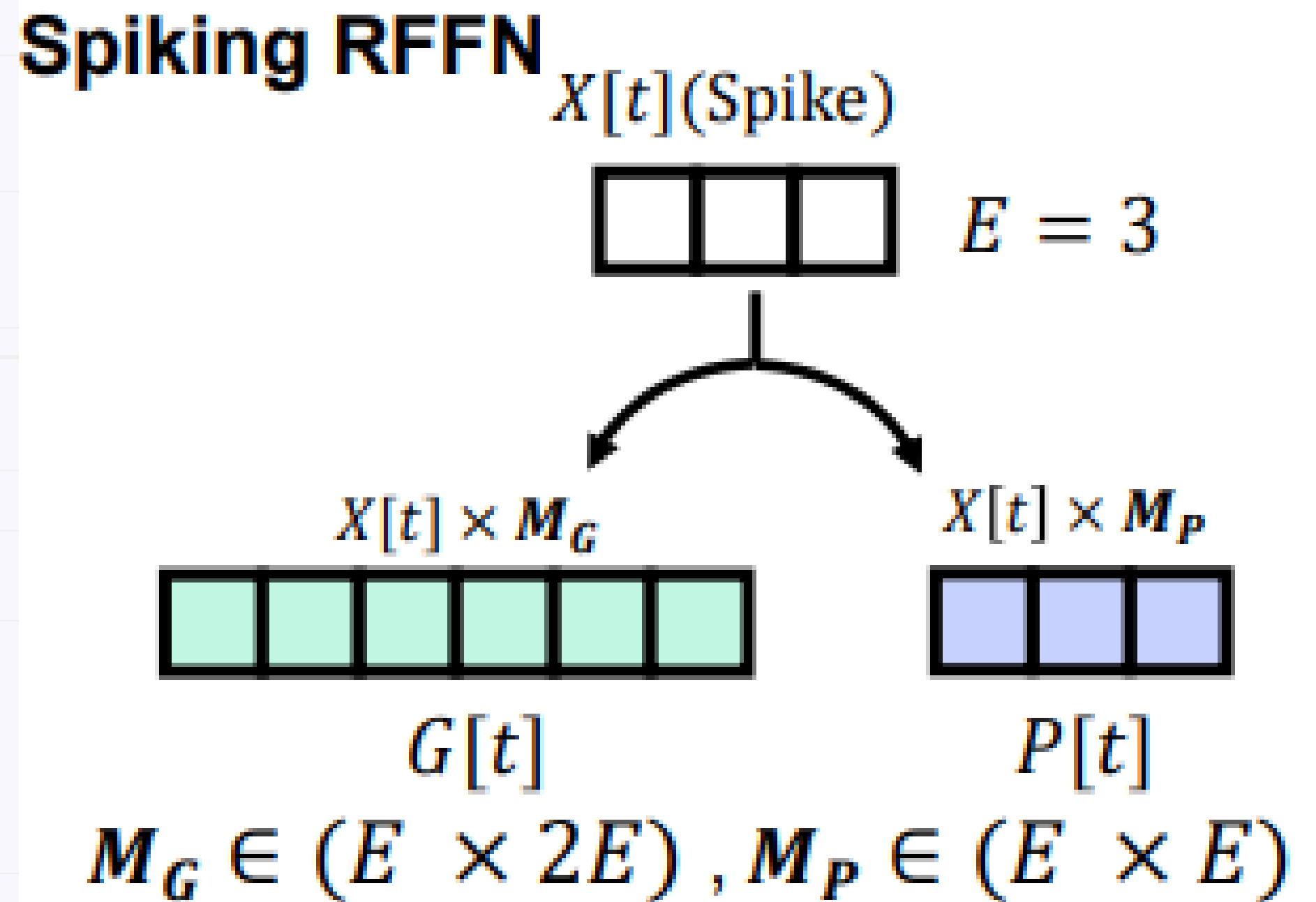
- **Chiến lược Suy giảm Liên tục (Continuous Decay Strategy):**
  - **SRWKV** không tự động điều chỉnh bản đồ chú ý theo đầu vào mà sử dụng các trạng thái ẩn  $A[t]$  và  $B[t]$  để lưu trữ thông tin quá khứ.
  - Thông tin quá khứ được kiểm soát bằng hai vector học được:  $Wd$  (làm giảm dần ảnh hưởng của thông tin cũ) và  $Wf$  (đánh trọng số cho thông tin hiện tại).
- **Không cần Masking:**
  - Trong quá trình sinh ngôn ngữ, **Self-Attention** cần che đi một nửa bản đồ chú ý để tránh rò rỉ thông tin từ tương lai.
  - **SRWKV** và **SNN**, với bản chất hồi quy một chiều tự nhiên, giải quyết trực tiếp vấn đề này mà không cần sử dụng Mask.

## 5. SPIKING RECEPTANCE FEED-FORWARD NETWORK (CHANNEL-MIXING)



Mô-đun SRFFN thực chất là một phiên bản được **xung hóa** (*spiking-adapted*) của **Gated Linear Unit (GLU)**. Khi nhận dữ liệu đầu vào  $\mathbf{X}[t]$  tại một bước thời gian, hệ thống chia luồng xử lý thành hai nhánh song song trước khi kết hợp chúng lại

### Spiking RFFN

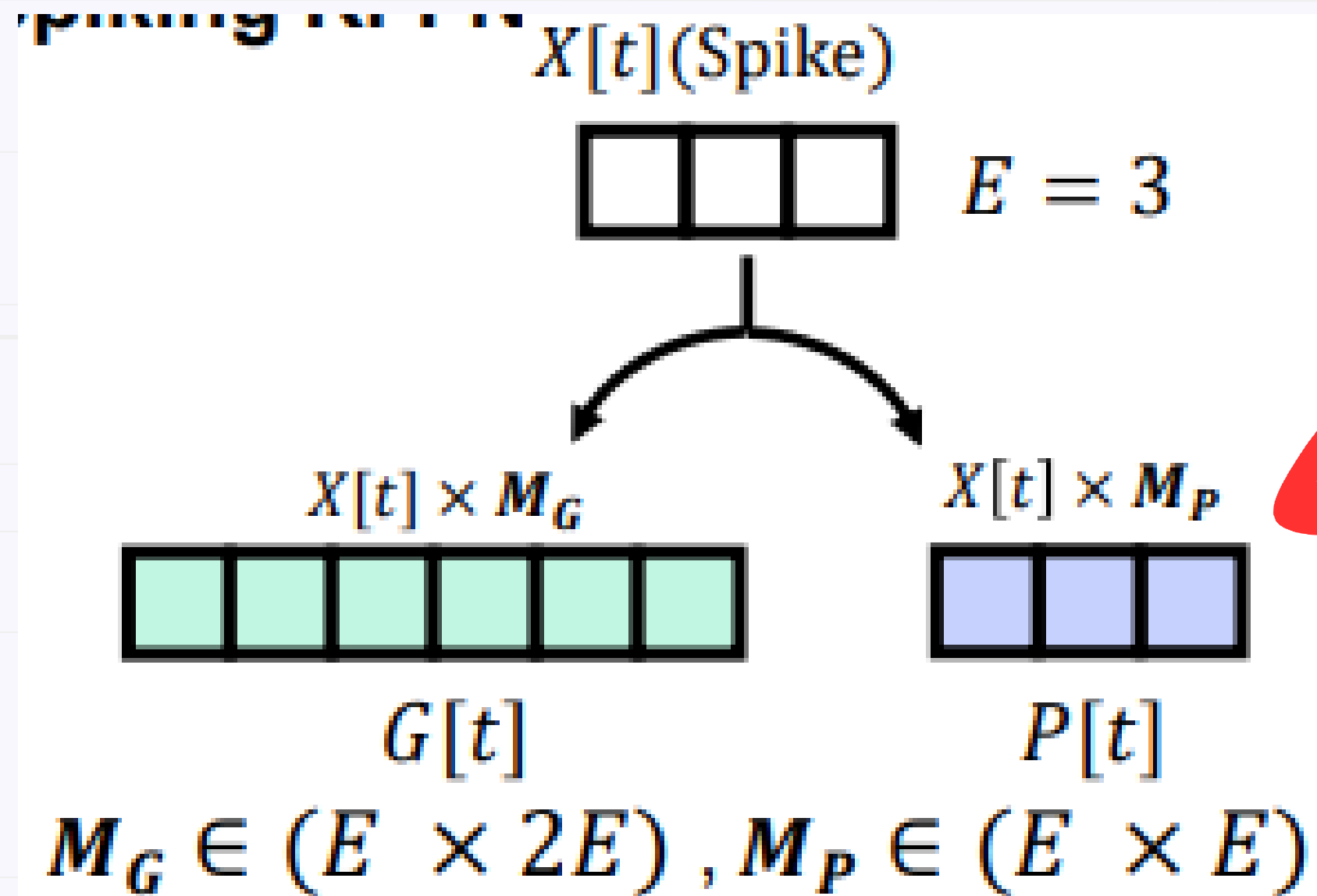


## 5. SPIKING RECEPTANCE FEED-FORWARD NETWORK (CHANNEL-MIXING)



$$Y'[t] = \mathcal{SN}(\sigma(M_P X[t]) \odot M_S(\text{Activation}(M_G X[t])))$$

**Nhánh 1 - Cổng kiểm soát (Gating Mechanism):** Tính toán bởi  $\sigma(M_P X[t])$   
Hàm Sigmoid  $\sigma$  tạo ra một ma trận các giá trị từ 0 đến 1. Nó đóng vai trò điều tiết lượng thông tin được phép chảy qua mạng. Từ *Receptance* trong tên gọi chính là cơ chế tiếp nhận này.



## 5. SPIKING RECEPTANCE FEED-FORWARD NETWORK (CHANNEL-MIXING)

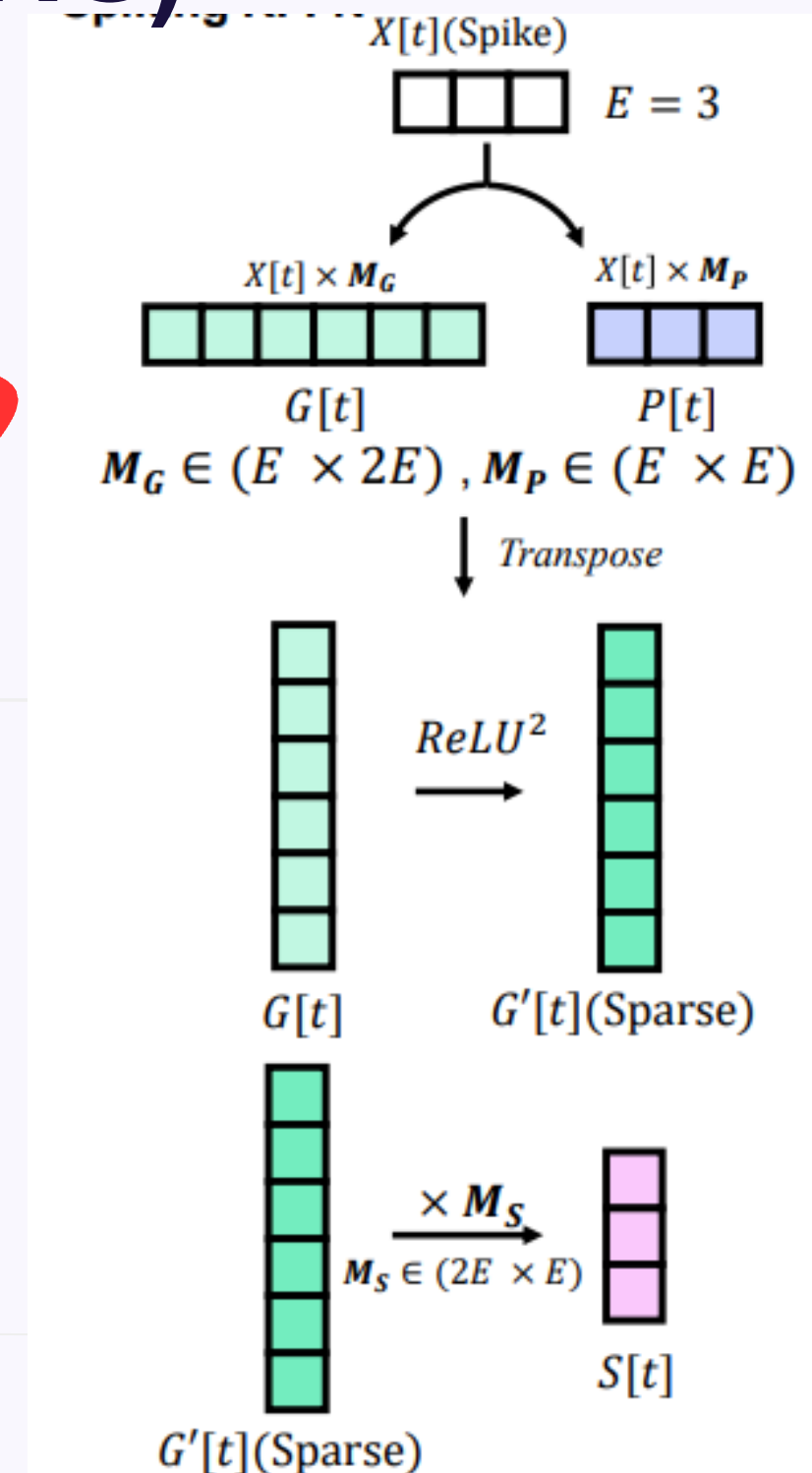


$$Y'[t] = \mathcal{SN}(\sigma(M_P X[t]) \odot M_S(\text{Activation}(M_G X[t])))$$

### Nhánh 2 - Xử lý thông tin chính:

Tính toán bởi  $M_S(\text{Activation}(M_G X[t]))$

Dữ liệu đi qua phép chiếu tuyến tính  $M_G$  sau đó đi qua hàm kích hoạt (thường là  $ReLU^2$  để giữ các giá trị dương và mượt hơn), và cuối cùng qua một lớp tuyến tính  $M_S$ .

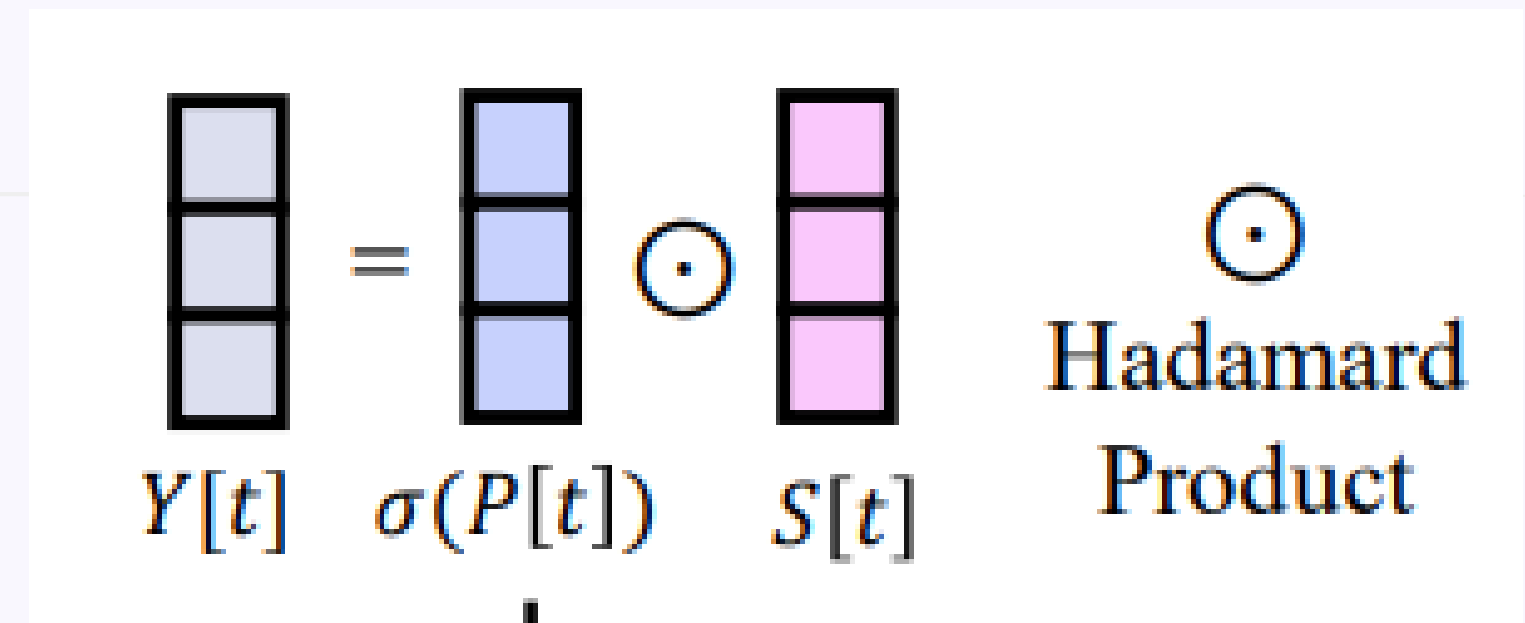


## 5. SPIKING RECEPTANCE FEED-FORWARD NETWORK (CHANNEL-MIXING)



$$Y'[t] = \mathcal{SN}(\sigma(M_P X[t]) \odot M_S(\text{Activation}(M_G X[t])))$$

**Kết hợp và Tạo Xung:** Kết quả của hai nhánh được nhân với nhau từng phần tử  $\odot$ .



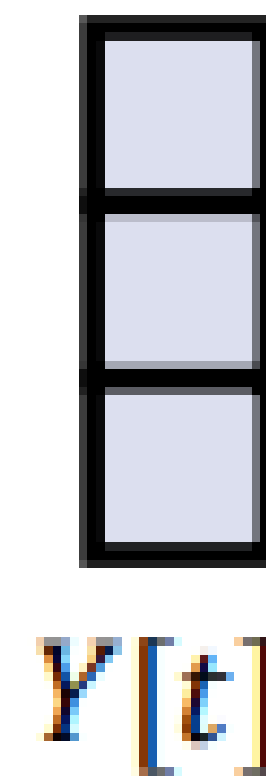


## 5. SPIKING RECEPTANCE FEED-FORWARD NETWORK (CHANNEL-MIXING)

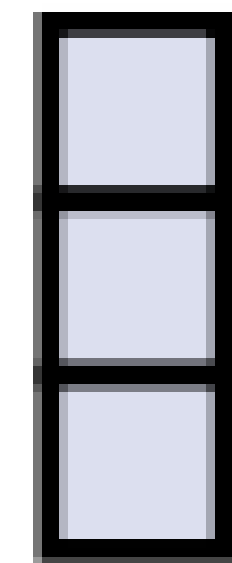


$$Y'[t] = \text{SN}(\sigma(M_P X[t]) \odot M_S(\text{Activation}(M_G X[t])))$$

- **Bộ lọc LIF:** Sau khi dữ liệu đi qua quá trình truyền thẳng trong **SRFFN**, đầu ra sẽ được nạp tiếp vào một **neuron LIF**.
- **Mục tiêu:** quyết định giúp mô hình bảo toàn các đặc điểm phát xung nhị phân ở cấp độ khối. Nhờ đó, nó **duy trì được độ thưa thớt** – tính chất thiết yếu giúp **SNN** đạt được hiệu quả năng lượng vượt trội.



Spiking  
Neuron



$O[t]$  (Spike)



*Đồ án Nhóm 5*

# ĐỒ ÁN CHATBOT AGENTIC TRONG QUẢN LÝ THƯƠNG MẠI

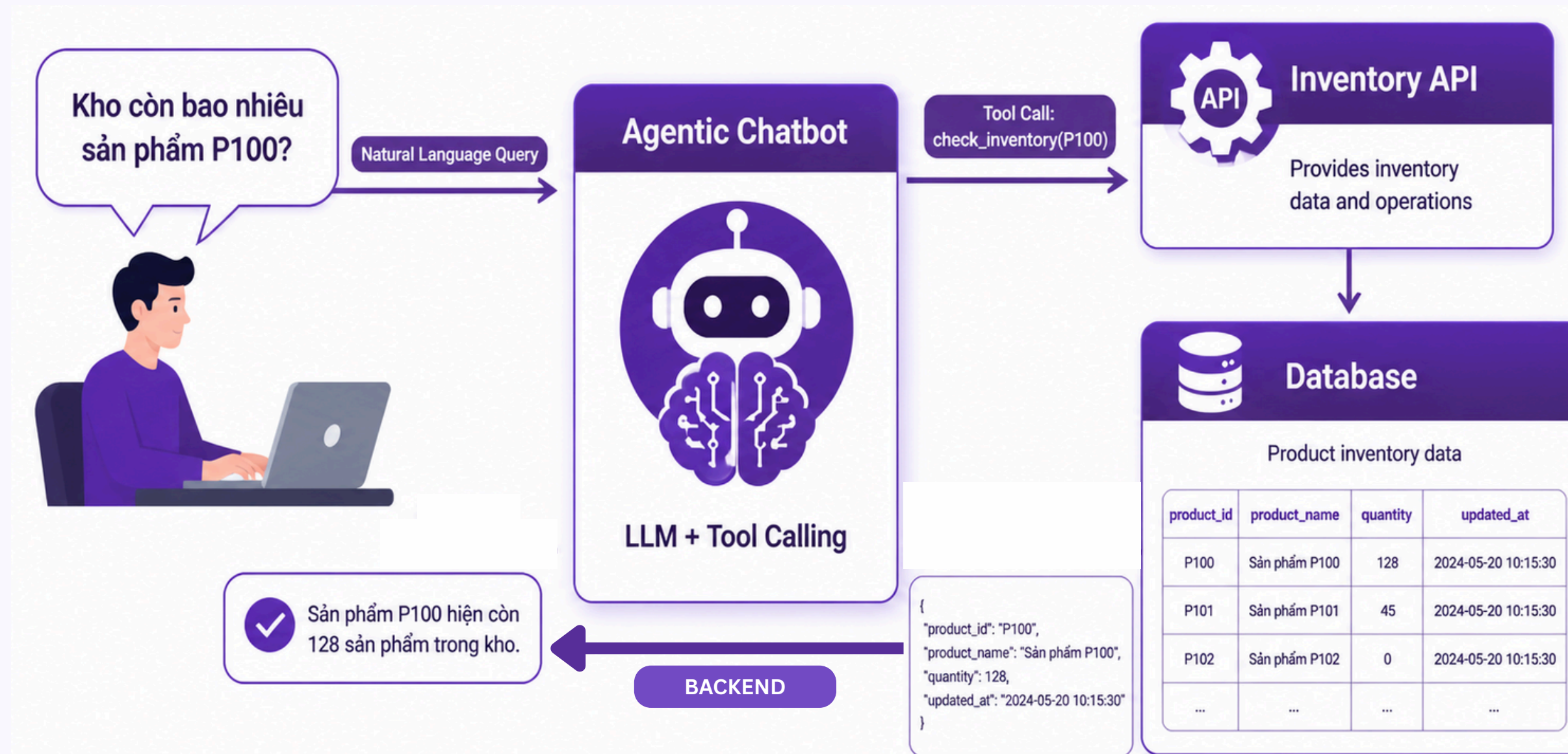
*23520984 - Phạm Quốc Nam*  
*23520662 - Nguyễn Khang Hy*



# 1. ĐẶT VẤN ĐỀ VÀ ỨNG DỤNG



- **Bài toán:** Quản lý kho, doanh thu, đơn hàng cần tra cứu CSDL liên tục. Các hệ thống dashboard tĩnh không linh hoạt. Người quản lý cần truy vấn bằng ngôn ngữ tự nhiên.
- **Giải pháp Agentic:** LLM phải đóng vai trò là một Agent:
  - Hiểu ý định → **Gọi đúng tool, đúng cấu trúc JSON.**
  - Trích xuất tham số → **Tool calling đúng tham số.**



## 2. KIẾN TRÚC HỆ THỐNG AGENTIC CHATBOT



- **User Input:** Người dùng nhập câu hỏi: "*Tồn kho của sản phẩm SP-990 là bao nhiêu?*"
- **SpikeGPT:** Mô hình suy luận và sinh ra một Tool Call được bọc trong cặp thẻ đặc biệt:

```
<tool_call>{"name": "check_inventory",  
"arguments": {"product_id": "SP-990"}}</tool_call>
```

- **Execution Engine (Backend):**
  - Hệ thống sử dụng Regex để bóc tách đoạn JSON nằm giữa thẻ `<tool_call>`.
  - Thực thi logic tương ứng thông qua **Supabase SDK** để truy vấn trực tiếp vào database đám mây.
- **Observation (Dữ liệu trả về):** Database trả về đối tượng dữ liệu đầy đủ (*bao gồm: name, stock, price*). Backend sẽ đóng gói lại thành một payload có cấu trúc.

**Finetune Ep220 + HeadQK** 

 9.63s

 ACTION / TOOL CALL

```
{  
  "name": "check_inventory",  
  "arguments": {  
    "product_id": "SP-990"  
  }  
}
```

 SYSTEM EXECUTION

```
{  
  "status": "success",  
  "product_name": "Tai nghe Bluetooth Sony WH-XB910N",  
  "stock": 42,  
  "price": "1,500,000 VND",  
  "message": "Sản phẩm Tai nghe Bluetooth Sony WH-XB910N còn 42 chỉ  
    ếc."  
}
```

XEM CHI TIẾT



### 3. SỬ DỤNG SPIKEGPT CHO AGENTIC TASK



- **Bảo mật dữ liệu thương mại:** Yêu cầu chạy cục bộ để tránh rò rỉ dữ liệu doanh thu.
- **Phù hợp thiết bị biên:** SpikeGPT tiết kiệm điện và bộ nhớ, có thể chạy trên các máy trạm cấu hình thấp tại nhà kho hoặc siêu thị nhỏ.
- **Thách thức của đồ án:** Khả năng suy luận nhị phân của SNN có đủ độ chính xác để thực hiện cấu trúc JSON nghiêm ngặt của Tool Calling hay không? Việc ghi nhớ và sinh ra đúng tham số từ câu input của người dùng có khả thi hay không?



# 4. PHƯƠNG PHÁP XÂY DỰNG TẬP DỮ LIỆU



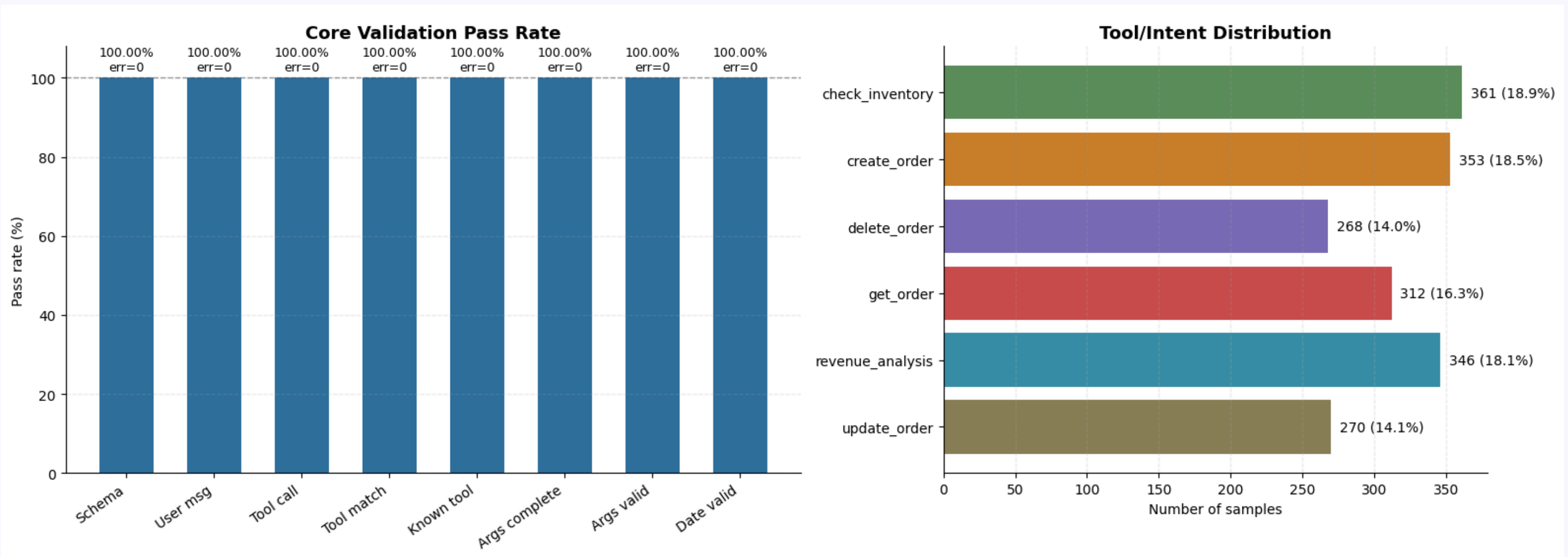
- Vì chưa có bộ dữ liệu Tool Calling chuyên cho Thương mại bằng Tiếng Việt, nhóm tự xây dựng.
- **Quy trình:**
  - a. Định nghĩa bộ 6 Tools quản lý.
  - b. Nhóm tự tay xây dựng một tập hợp các mẫu hội thoại hoàn hảo để làm chuẩn mực.
  - c. Dùng Gemini/GPT-4 để sinh hàng ngàn đoạn mô phỏng (User Agent Tool) dựa trên các mẫu dữ liệu đã tạo.
  - d. Làm sạch dữ liệu, loại bỏ ảo giác.
- **Kích thước tập dữ liệu: 1,910 mẫu** chất lượng cao.
- **Data Augmentation:** Từ 1,910 mẫu Augmentation thành:
  - 56k mẫu cho **tập Train**
  - 7,6k mẫu cho **tập Val**
  - 5k mẫu cho **tập Test**

|      | id         | description                                       | tool_used          | messages                                             |
|------|------------|---------------------------------------------------|--------------------|------------------------------------------------------|
| 0    | sample_001 | Kiểm tra tồn kho của một sản phẩm                 | [check_inventory]  | [{'role': 'user', 'content': 'Bạn kiểm tra giú...'}] |
| 1    | sample_002 | Tạo đơn hàng mới (Ví dụ gốc của bạn)              | [create_order]     | [{'role': 'user', 'content': 'Tôi muốn đặt 3 s...'}] |
| 2    | sample_003 | Thống kê doanh thu trong một khoảng thời gian     | [revenue_analysis] | [{'role': 'user', 'content': 'Cho tôi xem báo ...'}] |
| 3    | sample_004 | Tra cứu thông tin đơn hàng hiện tại               | [get_order]        | [{'role': 'user', 'content': 'Đơn hàng ORD-112...'}] |
| 4    | sample_005 | Cập nhật đơn hàng (thay đổi cả số lượng và địa... | [update_order]     | [{'role': 'user', 'content': 'Tôi muốn cập nhậ...'}] |
| ...  | ...        | ...                                               | ...                | ...                                                  |
| 1905 | 2240       | Cập nhật đơn hàng phụ kiện                        | [update_order]     | [{'role': 'user', 'content': 'Sửa đơn hàng mã ...'}] |
| 1906 | 2242       | Hủy đơn hàng do lỡ tay                            | [delete_order]     | [{'role': 'user', 'content': 'Hủy hộ cái đơn v...'}] |
| 1907 | 2250       | Hủy đơn hàng vì không cần nữa                     | [delete_order]     | [{'role': 'user', 'content': 'Xóa đơn hàng ORD...'}] |
| 1908 | 2256       | Hủy đơn hàng cuối cùng trong ngày                 | [delete_order]     | [{'role': 'user', 'content': 'Tôi muốn hủy yêu...'}] |
| 1909 | 2257       | Check mã vận đơn mới                              | [get_order]        | [{'role': 'user', 'content': 'Đơn hàng ORD1341...'}] |

## 5. TÍNH ĐÚNG ĐẲN CỦA DỮ LIỆU



- Toàn bộ **1.910 mẫu** dữ liệu gốc đều vượt qua các kiểm tra tính đúng: cấu trúc JSON, sự khớp giữa *tool\_used* và *tool\_call*, tính hợp lệ của *tool name*, *argument* bắt buộc và định dạng ngày, với tỉ lệ đạt 100%.
- Ngoài ra, phân bố dữ liệu trải đều trên 6 loại tool/intent, nên bộ dữ liệu đúng về mặt **schema và nhãn**, và có độ bao phủ tương đối cân bằng cho các tác vụ mà mô hình cần học.





## 6. CẤU TRÚC MẪU HỘI THOẠI



- Định dạng JSON từ Gemini/GPT-4 tạo ra:

```
{
  "id": "sample_575",
  "description": "Kiểm tra đơn bị hủy",
  "tool_used": ["get_order"],
  "messages": [
    {"role": "user", "content": "Đơn ORD-2211 của tôi sao nay chưa thấy tới?"},
    {"role": "assistant", "content": null, "tool_calls": [{"name": "get_order", "arguments": {"order_id": "ORD-2211"}}]},
    {"role": "tool", "content": "{\"order_id\": \"ORD-2211\", \"status\": \"Đã hủy do quá hạn thanh toán\"}"},
    {"role": "assistant", "content": "Đơn hàng ORD-2211 đã bị hệ thống tự động hủy do quá hạn thanh toán mất rồi ạ."}
  ]
},
```

- Sau đó parse lại thành dạng **JSONL** kèm **prompt** để SpikeGPT có thể học. Rồi tiếp tục **Tokenize** dữ liệu thành dạng nhị phân **bin/idx**

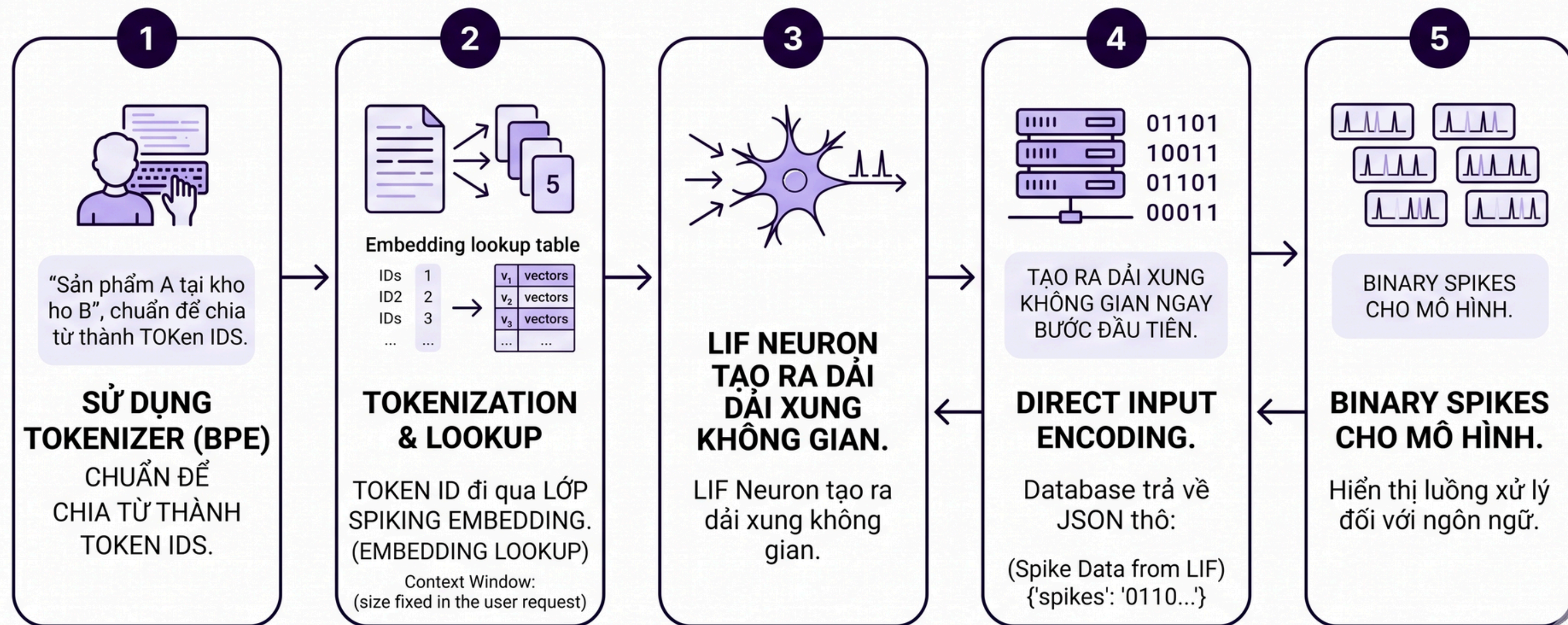
```
{"text": "<|im_start|>system\nHãy thực hiện theo yêu cầu<|im_end|>\n<|im_start|>user\nTính tổng thu nhập từ ngày 2029-02-27 đến 2027-12-12.<|im_end|>\n<|im_start|>system\nHãy thực hiện theo yêu cầu<|im_end|>\n<|im_start|>user\nCho cái đơn CODE-9447 dời về Số 156 Phố Cổ Dịch, Phường Ánh Sáng, Quận /<|im_end|>\n<|im_start|>system\nHãy thực hiện theo yêu cầu<|im_end|>\n<|im_start|>user\nUpdate lại mã M-6184 giùm, 98 món tới Số 76 Phố Ánh Sao, Phường 99, Huyện /<|im_end|>\n<|im_start|>system\nHãy thực hiện theo yêu cầu<|im_end|>\n<|im_start|>user\nCheck xem hóa đơn ID-2159 đã được gửi đi chưa.<|im_end|>\n<|im_start|>system\nHãy thực hiện theo yêu cầu<|im_end|>\n<|im_start|>user\nMua thêm 77 cái LAPTOP-73-MAX, ship hỏa tốc qua Số 520 Đường Số 8, Xã Bình Yên, Huyện /<|im_end|>\n<|im_start|>system\nHãy thực hiện theo yêu cầu<|im_end|>\n<|im_start|>user\nDoanh thu từ 2029-08-21 tới ngày 2027-10-12 là bao nhiêu?<|im_end|>\n<|im_start|>system\nHãy thực hiện theo yêu cầu<|im_end|>\n<|im_start|>user\nCheck tồn kho mã LAPTOP-21-PRO giúp chị.<|im_end|>\n<|im_start|>assistant\n<|im_end|>\n<|im_start|>system\nHãy thực hiện theo yêu cầu<|im_end|>\n<|im_start|>user\nGiao lệ 87 hộp MAC-54-PLUS tới Số 889 Đường Hoa Lan, Xã Bình Yên, Huyện /<|im_end|>\n<|im_start|>system\nHãy thực hiện theo yêu cầu<|im_end|>\n<|im_start|>user\nÊ bạn, xóa gấp giùm mình cái đơn ITEM-2466 nha.<|im_end|>\n<|im_start|>system\nHãy thực hiện theo yêu cầu<|im_end|>\n<|im_start|>user\nShop gửi 50 cái LAPTOP-16-X đến Số 151 Đường Bình Minh, Xã Bình Yên, Huyện /<|im_end|>\n<|im_start|>system\nHãy thực hiện theo yêu cầu<|im_end|>\n<|im_start|>user\nLên đơn 47 PC-51-PLUS về Số 339 Đường Số 8, Phường Ánh Sáng, Quận Alpha /<|im_end|>
```



# 7. TIỀN XỬ LÝ DỮ LIỆU CHO SPIKEGPT



- **Sử dụng tokenizer (BPE)** chuẩn để chia từ thành Token IDs.
- **Direct Input Encoding:** Không giống hình ảnh cần mã hóa rate-coding, với xử lý ngôn ngữ trên SpikeGPT, Token ID đi qua lớp Spiking Embedding (Embedding Lookup → LIF Neuron) để tạo ra dải xung không gian ngay bước đầu tiên.
- Kích thước chuỗi ngữ cảnh (Context Window) được fix ở mức **1024**.





## 8. CẤU HÌNH HUẤN LUYỆN (FINE-TUNING)



- **Mô hình gốc:** SpikeGPT-216M (*18 Layers, 768 Embedding Dim*).
- **Phương pháp:** Full Fine-tuning trên nền tảng SpikingJelly:
  - **BPTT (Backpropagation Through Time):** Lan truyền ngược qua 1024 bước thời gian.
  - **Surrogate Gradient:** Sử dụng hàm ArcTan để xấp xỉ đạo hàm cho nơ-ron xung.
- **Hyperparameters:**
  - **Optimizer:** Adam.
  - **Learning Rate:**  $1e-4$  với Cosine Annealing scheduler.
  - **Batch Size:** 2.
  - **Epochs:** Nhóm đã thử nghiệm các mốc 78 epochs và 220 epochs để so sánh độ hội tụ.
  - **HeadQK:** Tích hợp lớp HeadQK (256 dim) giúp tăng tỷ lệ trích xuất Arguments chính xác (vốn là điểm yếu cố hữu của kiến trúc RNN thuần).

## 9. TỐI ƯU HÓA HÀM MẤT MÁT



- Quá trình huấn luyện sử dụng **Causal Language Modeling (CLM)**, dự đoán token tiếp theo.
- Hàm mất mát **Cross-Entropy Loss** được tính toán tại lớp giải mã đầu ra (Readout Layer):

$$\mathcal{L} = - \sum_{t=1}^T \sum_{c=1}^V y_{t,c} \log(\hat{y}_{t,c})$$

- **Trong đó:**  $T$  là chiều dài chuỗi,  $V$  là kích thước từ vựng (Vocabulary size). Logits đầu ra được đưa qua Softmax để tạo phân phối xác suất trên từ vựng; Cross-Entropy đo sai lệch giữa phân phối dự đoán và token mục tiêu.
- **Hạn chế:** **Cross-Entropy** token-level tối ưu khả năng dự đoán token kế tiếp, nhưng chưa trực tiếp tối ưu Args Exact Match, nơi sai một ký tự trong tham số sẽ làm toàn bộ tool call bị tính sai.

## 9. TỐI ƯU HÓA HÀM MẤT MÁT

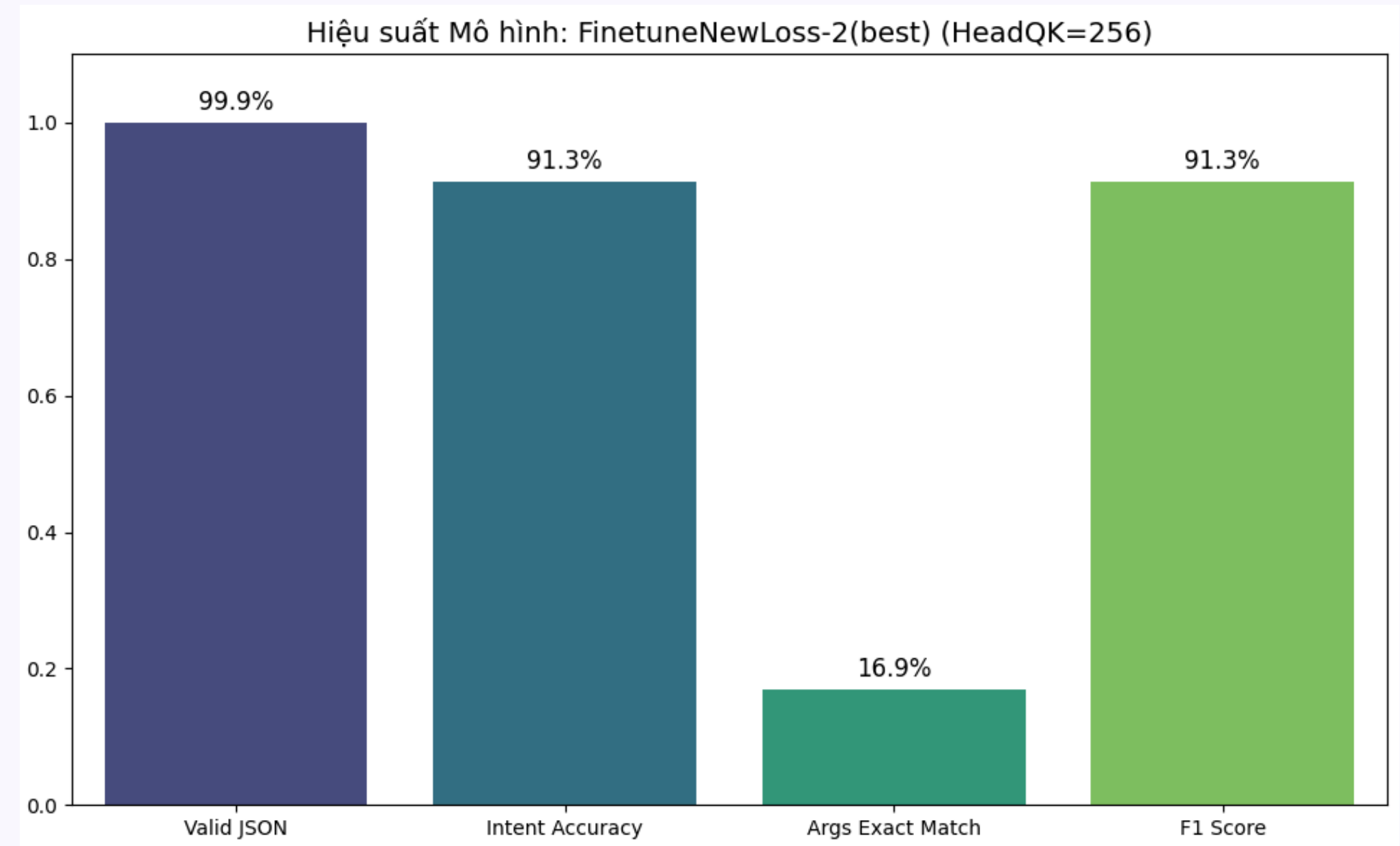


### Vấn đề:

- Tool Calling yêu cầu copy chính xác tham số: *order\_id*, *product\_id*, *quantity*, *address*.
- Cross-Entropy thường xem mọi token gần như ngang nhau.
- Sai một token trong argument làm Args Exact Match = sai toàn bộ

### Ý tưởng cải tiến:

- Giữ Causal LM Loss, nhưng tăng trọng số cho các token mục tiêu đã xuất hiện trong context.
- Các token này có khả năng là token cần copy từ câu hỏi người dùng.



**Kết quả:** Args Exact Match tăng từ **13.94%** lên **16.88%** (+2.94 điểm), nhưng Intent Acc và F1-score giảm nên nhóm không chọn làm mô hình chính.

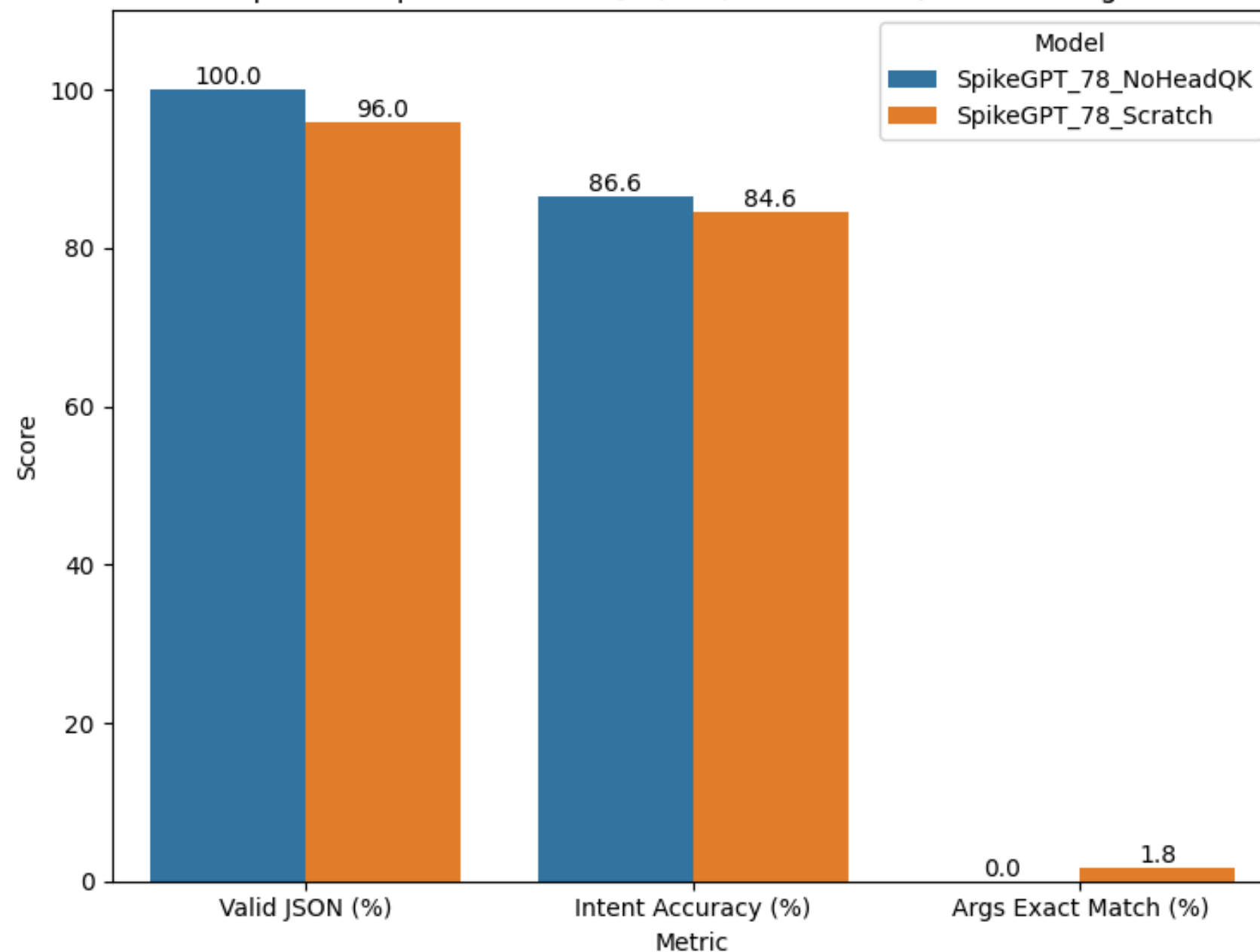


# 10. KẾT QUẢ THỰC NGHIỆM

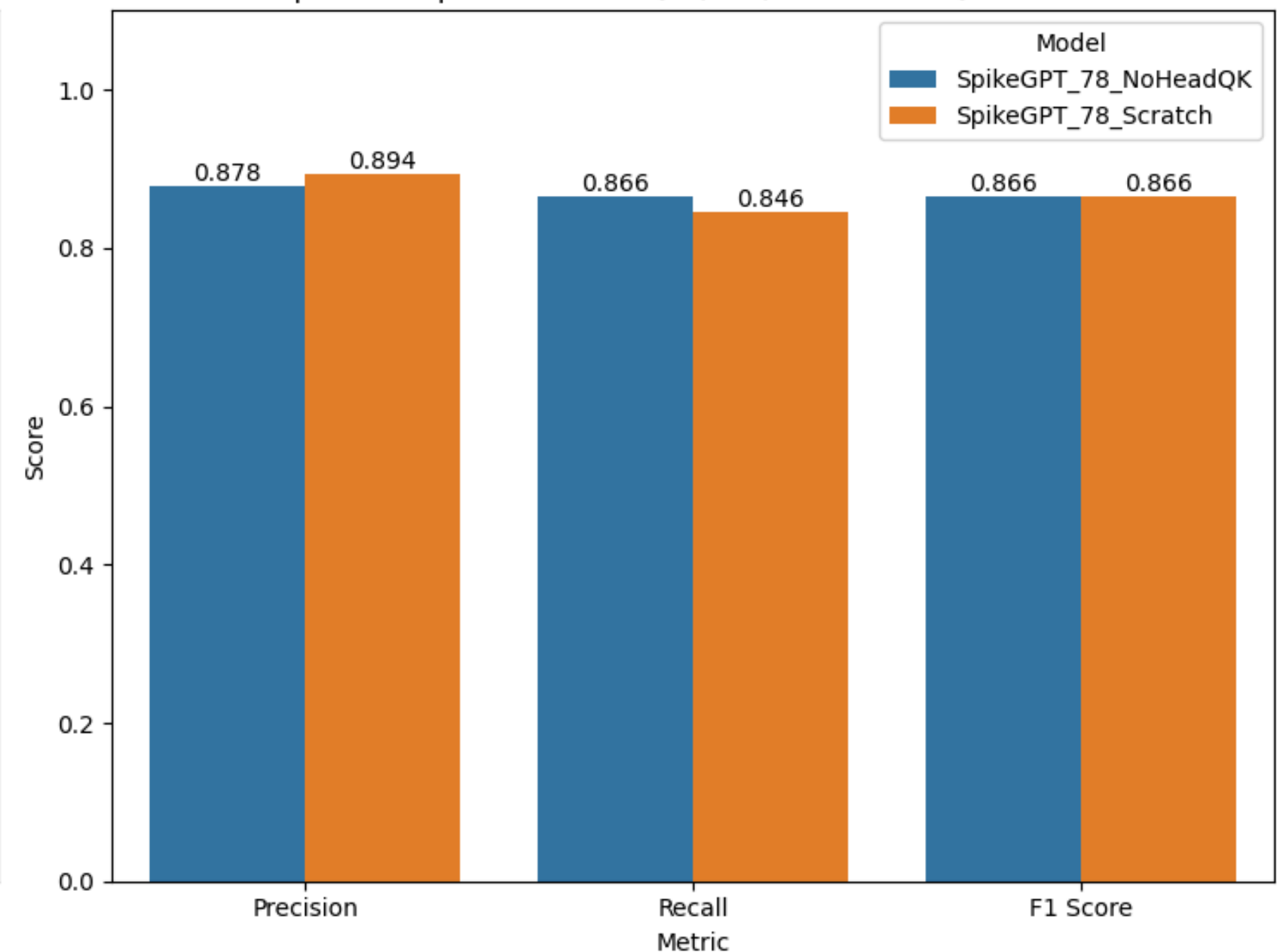


- Bật HeadQK cho bài toán tool-calling nhằm hỗ trợ copy token, giúp **tăng xác suất** copy tham số từ ngữ cảnh, nhưng **chưa** giải quyết triệt để bài toán **Args Exact Match**, trong khi paper gốc tập trung vào language modeling/NLU chứ không tối ưu riêng cho exact argument copying.

SpikeGPT Epoch 78: HeadQK (256) vs No HeadQK - Percentages



SpikeGPT Epoch 78: HeadQK (256) vs No HeadQK - Scores



# 10. KẾT QUẢ THỰC NGHIỆM



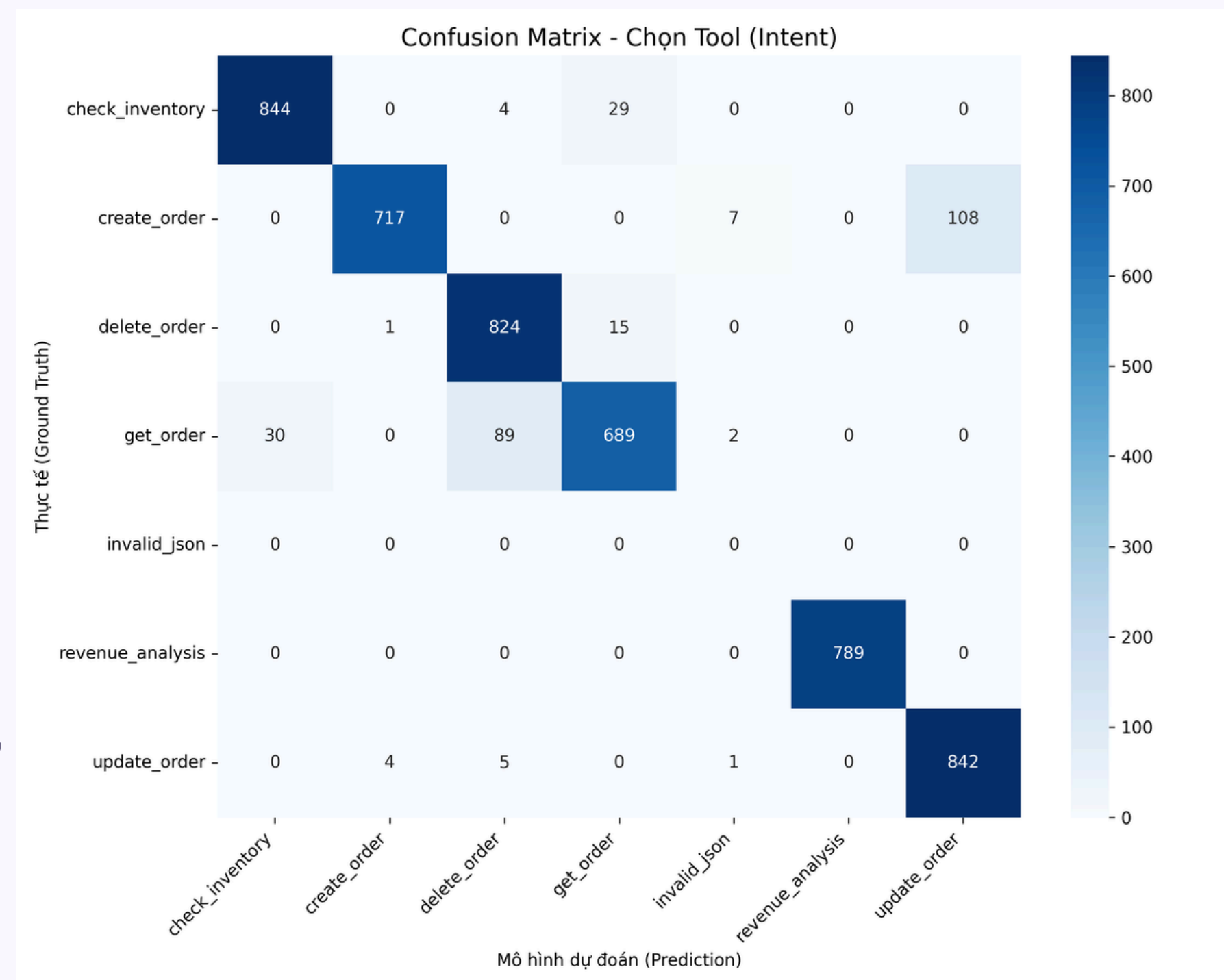
- So sánh SpikeGPT ở các mức Epoch khác nhau và Baseline (GPT-2 Small/Medium).
- **Bảng kết quả:**

| Model                         | Valid JSON (%) | Intent Accuracy (%) | Args Exact Match (%) | Precision | Recall | F1 Score |
|-------------------------------|----------------|---------------------|----------------------|-----------|--------|----------|
| SpikeGPT NoHeadQK (epoch 78)  | 100.00         | 86.62               | 0.00                 | 0.8781    | 0.8662 | 0.8659   |
| SpikeGPT Scratch (epoch 78)   | 95.98          | 84.64               | 1.78                 | 0.8944    | 0.8464 | 0.8655   |
| SpikeGPT Scratch (epoch 220)  | 91.52          | 81.24               | 4.68                 | 0.9022    | 0.8124 | 0.8463   |
| SpikeGPT Finetune (epoch 220) | 99.80          | 94.10               | 13.94                | 0.9459    | 0.9410 | 0.9415   |
| BASELINE (Finetune)           |                |                     |                      |           |        |          |
| GPT2_Small (124M)             | 99.98          | 96.52               | 68.30                | 0.9671    | 0.9652 | 0.9648   |
| GPT2_Medium (355M)            | 100.00         | 96.78               | 67.76                | 0.9696    | 0.9678 | 0.9678   |

# 11. PHÂN TÍCH VÀ THẢO LUẬN KẾT QUẢ



- **Thành tựu:** SpikeGPT (SNN) đạt được độ chính xác nhận diện ý định (*Intent Accuracy*) lên tới **94.10%**, và khả năng sinh mã JSON hợp lệ gần như tuyệt đối (**99.8%**). Điều này chứng minh mạng xung hoàn toàn có đủ năng lực logic để cấu trúc dữ liệu khắt khe của Tool Calling.
- **Điểm nghẽn:** Chỉ số Args Exact Match (trích xuất chính xác tham số) còn thấp (**13.94%**) so với ANN GPT-2 (**68%**).
- **Giải thích:** Quá trình ép dữ liệu từ số thực qua hàm **Heaviside (0/1)** tạo ra sự mất mát thông tin cục bộ. SNN rất giỏi **Classification**, nhưng gặp khó khăn khi trích xuất chuỗi ký tự chính xác tuyệt đối (Extraction).
- **Args Exact Match là metric nghiêm ngặt:** sai một ký tự, thiếu dấu gạch ngang hoặc sai một phần địa chỉ đều tính là **sai toàn bộ**.



SpikeGPT Finetune Ep220 + HeadQK

## 12. DEMO HỆ THỐNG THỰC TẾ



- **Phân tích & Trích xuất:** SpikeGPT đọc yêu cầu, xác định công cụ phù hợp. Nhờ cơ chế HeadQK, mô hình trích xuất chính xác các tham số trực tiếp từ ngữ cảnh.
- **Sinh mã định dạng:** Mô hình sinh câu trả lời kèm thẻ định dạng để gọi hàm:  
`<tool_call>{"name": "check_inventory", "arguments": {"product_id": "SP-101"}}</tool_call>`
- **Thực thi & Phản hồi:** Hệ thống Parser bóc tách mã JSON → Truy vấn Database thực tế → Trả kết quả hiển thị trực quan cho người dùng

SpikeGPT Scratch Ep220 + HeadQK

↺

Đã chuyển sang mô hình **SpikeGPT Scratch Ep220 + HeadQK**. Lịch sử đã được xoá.

kiểm tra mã SP-990 còn hàng hay không?

↻

Sản phẩm Tai nghe Bluetooth Sony WH-XB910N còn 42 chiếc.

Trạng thái sản phẩm

Sản phẩm:

Tai nghe Bluetooth Sony WH-XB910N

Tồn kho:

Còn hàng (42)

Giá:

1,500,000 VND

Nhập yêu cầu từ người dùng...

➤

# 13. HẠN CHẾ CỦA ĐỒ ÁN



## 1. Rào cản phần cứng:

- **Thực trạng:** Hiện tại SpikeGPT đang được **giả lập** trên phần cứng GPU truyền thống (Nvidia T4).
- **Hệ quả:** Lợi ích cốt lõi của SNN là tính toán dựa trên sự kiện (Event-driven) và tiết kiệm năng lượng **chỉ mang tính lý thuyết**.
- **Hướng đi:** Cần triển khai trên các chip chuyên dụng **Neuromorphic (như Intel Loihi)** để đạt được hiệu suất năng lượng thực tế.

## 2. Sự đánh đổi giữa Hiệu năng và Độ chính xác

- **Vấn đề trích xuất thực thể:** Khả năng bắt các tham số (ID, ngày tháng) của SpikeGPT còn yếu hơn Transformer do bản chất **Quantization (lượng tử hóa)** khắc nghiệt của các xung nhị phân, gây mất mát thông tin so với các giá trị số thực độ phân giải cao của GPT-2.
- **Sự phức tạp hóa:** Việc thêm **HeadQK** giúp cải thiện độ chính xác nhưng vô tình tăng độ phức tạp  $O(T^2)$  ở **lớp cuối** (các lớp trước đó vẫn là  $O(T)$ ), làm giảm ưu thế về tốc độ xử lý câu dài của kiến trúc SNN.

## 3. Khoảng cách về sự hội tụ (Convergence Gap)

- **So sánh:** Với cùng một khoảng kích thước tham số (216M so sánh với 125M/355M) và lượng dữ liệu fine-tuning, SpikeGPT vẫn cần nhiều Epochs hơn và có độ ổn định thấp hơn so với GPT-2 trong các tác vụ logic phức tạp.



# 14. HƯỚNG PHÁT TRIỂN & KẾT LUẬN



## 1. Hướng phát triển:

- **Kiến trúc lai Hybrid SNN-ANN:** Tận dụng ANN (Attention) ở các tầng đầu để đảm bảo độ chính xác tuyệt đối trong việc trích xuất thực thể, đồng thời sử dụng SNN cho các tầng ngữ cảnh sâu để duy trì lợi thế tiết kiệm năng lượng.
- **Tích hợp cơ chế RAG (Retrieval-Augmented Generation):** Kết nối mô hình với hệ thống tri thức nội bộ (PDF, Quy trình doanh nghiệp), giúp Agent không chỉ gọi tool mà còn có thể tư vấn chuyên sâu dựa trên tài liệu thực tế.
- **Triển khai trên phần cứng chuyên dụng:** Đưa mô hình chạy thực tế trên chip Neuromorphic để đo đạc chỉ số tiêu thụ điện năng so với kiến trúc Transformer truyền thống.

## 2. Kết luận:

- **Tiên phong công nghệ:** Đề án đã hiện thực hóa thành công quy trình **Agentic Chatbot** trên nền tảng SpikeGPT – một kiến trúc mạng nơ-ron mô phỏng não người cực kỳ mới mẻ và đầy thách thức.
- **Tính khả thi:** Dù còn những điểm hạn chế về độ chính xác trích xuất tham số, hệ thống đã chứng minh được khả năng tương tác thời gian thực với Database và thực thi công cụ một cách logic.
- **Tầm nhìn:** Đây là bước đệm quan trọng hướng tới kỷ nguyên **Green AI** – nơi các hệ thống trí tuệ nhân tạo mạnh mẽ có thể vận hành bền bỉ trên các thiết bị Edge mà không tiêu tốn quá nhiều tài nguyên năng lượng.

# THANK YOU

